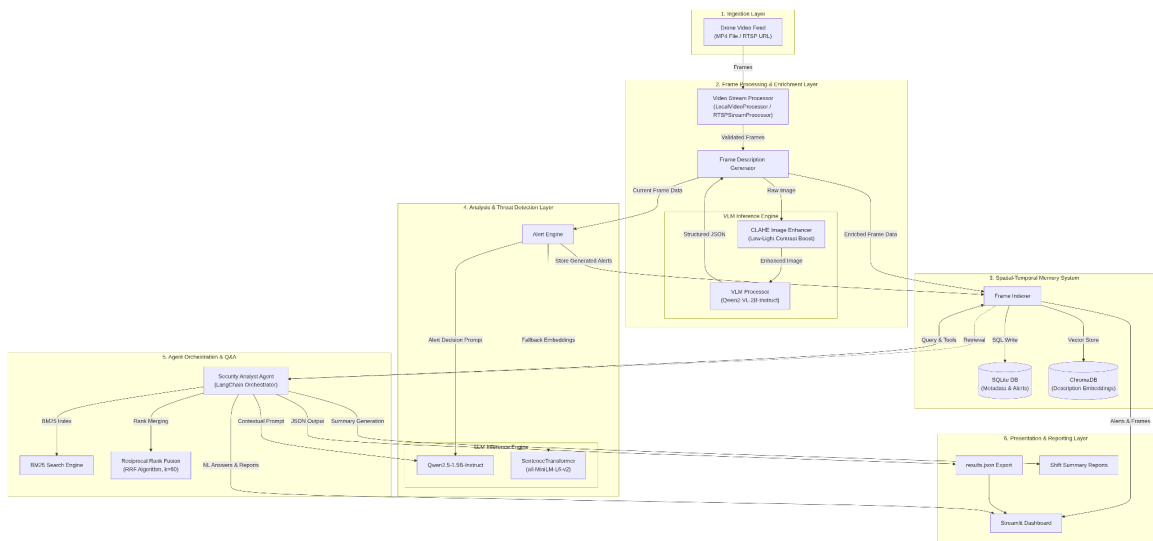


1. Executive Summary

The Drone Security Analyst Agent is an intelligent, edge-capable property security monitoring system that analyzes live drone video feeds (RTSP or local video files), indexes detections frame-by-frame, triggers real-time security alerts, and supports conversational natural-language Q&A.



Vision Model	Qwen2-VL-2B-Instruct — structured JSON frame analysis with CLaHE low-light enhancement
Reasoning LLM	Qwen2.5-1.5B-Instruct — alert validation, threat scoring, and retrieval-augmented Q&A
Memory Store	Hybrid SQLite (relational/temporal) + ChromaDB (semantic vector) indexing
Retrieval	BM25 keyword + ChromaDB vector search merged via Reciprocal Rank Fusion (RRF, k=60)
Orchestration	Custom Python Security Agent with manual prompt formatting and string operations
Dashboard	Streamlit UI — static log viewer for post-run analysis and alert review

Through rapid development iterations, this project evolved from a basic two-step frame-description prototype into a functioning pipeline capable of running on a single consumer GPU, utilizing entirely custom Python logic for orchestration.

2. Feature Specification

Value Proposition

The Drone Security Analyst Agent enhances property security with automated monitoring. Property owners receive intelligent, context-aware alerts rather than raw video footage, dramatically reducing manual review time while establishing a searchable history of events.

Key Requirements

REQ-1: Real-Time Detection	The system must analyze incoming drone video at configurable frame rates (0.5-2 FPS), generate structured scene descriptions, and detect objects (vehicles, persons).
REQ-2: Intelligent Alerting	Alerts evaluate contextual scenarios using prototype heuristics (e.g., dwell time limits and repeat visitor tracking). The current engine serves as a baseline for future accuracy tuning and false-positive reduction.
REQ-3: Queryable History	All analyzed frames must be persistently indexed by timestamp, location, and detected objects. The system must support both exact relational queries (e.g., frames at Gate A after 23:00) and fuzzy semantic searches (e.g., show suspicious nighttime activity).

3. System Architecture & Design

End-to-End Pipeline

The system operates as a streaming pipeline from video ingestion through to user-facing dashboards. Each stage is decoupled, allowing independent scaling.

Step	Component	Description
1	Video Stream Input	MP4 / RTSP -> OpenCV threaded reader with MPEG-aware full-decode
2	Frame Extraction	Configurable FPS sampling; populates metadata with video stream statistics (source, total_frames, etc.)
3	VLM Analysis	CLAHE contrast enhancement -> Qwen2-VL-2B -> structured JSON (description, objects, activity_type)
4	Hybrid Indexing	SQLite (relational metadata) + ChromaDB (sentence-transformer embeddings)
5	Alert Engine	Qwen2.5 LLM evaluation (dual-invocation for decision & enrichment) -> temporal heuristics -> embedding fallback
6	Security Agent	Custom Python orchestrator -> BM25 + vector search -> RRF fusion -> top-10 context -> LLM answer synthesis
7	Streamlit Dashboard	Static viewer: loads results.json for post-run alert logs, metrics, and frame proofs

Component Map

File	Component	Responsibility
src/video_stream.py	Video Reader	Thread-safe OpenCV stream; MPEG-aware full-decode sampling
src/vlm_processor.py	VLM Processor	Qwen2-VL-2B; structured JSON output; CLAHE image enhancement

src/frame_description.py	Frame Description	Bridge from raw frame to FrameDescription via VLM
src/frame_indexer.py	Frame Indexer	SQLite + ChromaDB persistence and query layer
src/alert_engine.py	Alert Engine	LLM decision & enrichment pipeline with embedding fallback
src/agent.py	Security Agent	Custom Python class (No LangChain); BM25 + vector RRF hybrid search; Q&A
src/live_pipeline.py	Live Pipeline	End-to-end video-to-JSON orchestrator with frame image export
src/dashboard.py	Dashboard	Streamlit UI: static viewer for pre-computed pipeline results
src/config.py	Config	All tunable parameters and static metrics (e.g., confidence scores) as dataclasses
prompts/	Prompt Templates	Centralized VLM, agent Q&A, alert decision, enrichment prompts

4. Justification of Tools & Configurations

4.1 VLM Selection: Qwen2-VL-2B-Instruct vs. BLIP-2

Problem	The initial specification suggested BLIP-2. However, BLIP-2 produced simplistic captions (e.g., a truck on the road), lacked spatial awareness, and frequently echoed prompt prefixes. Critically, BLIP-2 cannot produce structured JSON directly.
Solution	Qwen2-VL-2B-Instruct was integrated. It supports Native Dynamic Resolution and excels at instruction-following, allowing a structured JSON output (description, objects, activity_type) in a single inference call.

4.2 LLM Selection: Qwen2.5-1.5B-Instruct vs. Gemma-2B / SmolLM2-1.7B

Problem	The project iterated through Gemma-1.1-2b-it and SmolLM2-1.7B-Instruct. Both struggled to consistently follow strict JSON formats or evaluate complex security heuristics under small parameter budgets.
Solution	Qwen2.5-1.5B-Instruct represents the state-of-the-art for micro-LLMs in instruction following, JSON formatting, and conversational reasoning.

4.3 Hybrid Memory: SQLite + ChromaDB

Problem	Pure vector databases excel at semantic similarity but are slow at structured filters (e.g., frames between 23:00 and 02:00). Pure relational databases cannot perform semantic or fuzzy search.
Solution	Combining SQLite (fast indexed temporal queries) with ChromaDB (all-MiniLM-L6-v2 embeddings) provides both capabilities. Reciprocal Rank Fusion (RRF, k=60) merges ranked results from both systems.

4.4 Retrieval: Reciprocal Rank Fusion (RRF)

Problem	Keyword (BM25) search misses semantic intent — automobile misses car. Vector
----------------	--

	search misses exact terms — F150 may not match well semantically.
Solution	RRF scores documents by their rank across both search methods. Neither pure approach is relied upon exclusively, giving balanced recall for both precise and fuzzy queries.

5. Design Assumptions

A1	Dataset Choice	Development and testing used sample_data. This footage contains a realistic mix of daytime vehicle movements, perimeter views, and potential security scenarios.
A2	Edge Hardware Constraints	The system targets a single desktop/server GPU. Models were constrained around a 2B parameter, so both co-exist in VRAM without OOM crashes.
A3	Conservative Alert Bias	False positives are highly disruptive. The alert engine's LLM prompt instructs Qwen2.5 to act conservatively: normal daytime vehicle or pedestrian activity is suppressed.
A4	RTSP Simulation	For development, local .mpg and .mp4 files with --loop were used to simulate a continuous RTSP stream. The VideoStreamProcessor is architected identically for live RTSP and local files.

6. Development Challenges & Solutions

C1 - vLLM VRAM Domination & OOM Crashes

Problem	Early iterations used vLLM to serve Qwen2-VL for faster batch inference. By default, vLLM pre-allocates up to 90% of GPU VRAM for KV-caching, causing CUDA OOM errors.
Solution	vLLM was removed in favor of HuggingFace Transformers. Explicit pixel budget constraints were applied in <code>vlm_processor.py</code> .

C2 - OpenCV Frame Seeking & MPEG Decoder Corruption

Problem	<code>cap.set()</code> was too slow. Refactoring to <code>cap.grab()</code> for fast forward-skipping completely corrupted MPEG-1/2 files.
Solution	Conditional full-decode logic was implemented. MPEG sources decode every frame via <code>cap.read()</code> and discard non-sampled frames via counter.

C3 - Forced FFMPEG Backend Producing Rainbow Static

Problem	Forcing <code>cv2.CAP_FFMPEG</code> on MPEG files caused the decoder to output raw YUV bytes without BGR/RGB conversion, producing rainbow static frames.
Solution	The forced backend flag was removed. A frame validator <code>_is_valid_frame()</code> was added.

C4 - Low-Light & Contrast Failures

Problem	Drone video at twilight/night suffers from low contrast and noise, causing the VLM to miss people or vehicles entirely.
Solution	CLAHE (Contrast Limited Adaptive Histogram Equalization) was applied in the image preparation step via LAB color space conversion.

C5 - Markdown JSON Wrapping in VLM Output

Problem	Qwen2-VL occasionally wrapped JSON responses in Markdown code blocks despite instructions to output raw JSON, crashing <code>json.loads()</code> .
---------	--

Solution	Robust JSON parsing was implemented: strips markdown wrappers, then locates bounding braces { ... } via substring search.
----------	---

C6 - Alerting Heuristic Baseline Constraints

Problem	Initial heuristics for dwell time and repeat visitors trigger false positives due to generic object matching (treating any 'vehicle' as a repeat visit) and unscaled temporal thresholds (e.g., 240 mins).
Solution	Identified for future refactoring: requires using specific object identifiers (e.g., 'blue Ford f150') and normalizing time thresholds to match shorter flight video scales.

7. Results & Example Outputs

7.1 VLM Frame Analysis Output

When processing a frame showing a vehicle near the perimeter fence, the VLM produces a clean structured payload in a single inference call:

```
{
  "description": "A white delivery van is driving slowly along
the gravel road near the perimeter fence.",
  "objects": ["vehicle", "road", "nature"],
  "activity_type": "vehicle"
}
```

7.2 Alert Generation (Example)

Alert Type	Severity	Confidence Context	Message
loitering_midnight	HIGH	Static Configuration	Detected a person standing near the main gate entrance during nighttime hours (23:45:00).
repeat_visitor	MEDIUM	Static Configuration	Repeat visit: Ford F150 entered at 2026-06-14 08:00:00 and 2026-06-14 14:30:00.
perimeter_breach	HIGH	Static Configuration	Unknown individual detected in restricted perimeter zone at 01:15:00.

7.3 Retrieval-Augmented Q&A Example

User Question	How many vehicles appeared more than once today?
Retrieval	RRF fusion retrieved 4 relevant frames from BM25 + ChromaDB search.
Agent Response	Based on the security logs, a single vehicle (a blue Ford F150 pickup truck) appeared twice today. It was first logged entering the main gate at 08:00:02 (Frame #120) and seen again at the north gate road at 14:30:15 (Frame #780).

8. AI Assistance Retrospective

AI pair-programming tools such as anti-gravity and github copilot were utilized to accelerate the development pipeline. The following table documents specific contributions and the human customizations applied:

Area	AI Contribution	Human Customization
Agent Orchestration	Scaffolded custom Python agent framework and conversational history string management	Refactored away from LangChain; tuned context window management for security-specific Q&A
Prompt Engineering	Generated initial VLM and alert decision prompts; iterated on JSON schema enforcement	Refined dual-invocation prompt logic (decision + enrichment) for off-hour scenarios
OpenCV Debugging	Diagnosed inter-frame dependency issue in MPEG files; suggested grab() vs read() logic	Implemented conditional full-decode path and frame validation heuristics
Streamlit UI	Generated boilerplate for static log viewing and metric display	Customized JSON parser to load and visualize pre-computed pipeline results
Database Layer	Scaffolded SQLite schema and ChromaDB collection setup	Optimized indices for timestamp/location queries and tuned RRF fusion parameters
CLAHE Enhancement	Suggested LAB colorspace CLAHE approach for low-light improvement	Tuned threshold ($avg_l < 130$) and clip limit for aerial drone footage characteristics

9. Future Improvements

The following architectural upgrades would be prioritized if submission time were not constrained:

Video Clip Summarization	HIGH	Integrating a video-LLM (Video-LLaVA or Qwen2-VL-7B) to process 10-second clips would enable detecting complex behaviors such as fence-climbing or erratic driving.
Advanced VLM via Triton	HIGH	Migrating to Qwen2-VL-7B served via NVIDIA Triton Inference Server would improve detection accuracy in low-light and long-distance aerial shots.
Heuristics Refactoring	HIGH	Updating the alert engine to use dynamic parameter matching (e.g., 'blue Ford F150') and normalized thresholds rather than generic object matching.
Cross-Session Memory	MEDIUM	Integrating Redis + vector cache would allow investigators to resume a conversation about an incident across shifts.
Confidence Calibration	LOW	Allow security operators to confirm or dismiss alerts via the dashboard to fine-tune the alert decision prompts.

10. Scaling Recommendations

Layer	Current (Prototype)	Production Scale
Database	SQLite (single-writer)	PostgreSQL + TimescaleDB (multi-tenant, time-series)
Vector Store	ChromaDB (embedded)	pgvector or Qdrant (dedicated vector service)
VLM Inference	HuggingFace Transformers (single GPU)	vLLM / NVIDIA Triton (multi-GPU, batched)
Frame Ingestion	OpenCV thread queue (single stream)	Apache Kafka (multi-camera, buffered ingestion)

Deployment	Single machine, Python venv	Docker + Kubernetes, GPU node pool
------------	--------------------------------	---------------------------------------

The prototype architecture is deliberately modular, enabling confident scaling from a single-drone prototype to a multi-site, multi-drone production deployment.

10. Links

1. System Diagrams:
https://drive.google.com/drive/folders/1zp4piSwvzBp_MLXLYmiaHCdxZ71WkA6t?usp=sharing
2. Explanation Video:
https://drive.google.com/file/d/1XssemQmgx5S1zGfMKJPmNnwenGRnj99k/view?usp=drive_link
3. Sample Test Video:
https://drive.google.com/file/d/12eV-Smt11YhfO8gEH4W-Z9dYmfyKtPr_/view?usp=drive_link